AWS VPC PEERING PROJECT

FINAL MINI PROJECT REPORT

Amazon Web Services Internship
EWB EDUTECH PRIVATE LIMITED

DETAIL	INFORMATION
STUDENT	MD AQUIB ANSARI
COLLEGE	BABU BANARASI DAS UNIVERSITY LUCKNOW
INTERNSHIP PROGRAMME	Amazon Web Services Internship
ROLE	AWS Cloud Intern
OFFER LETTER ID	EWB/OL/2026/AWS/0235
INTERNSHIP PERIOD	28 September 2026 · 4 Weeks
MODE / STIPEND	Virtual / Pan India · Unpaid
EMAIL	mdaquib45c@bbdu.ac.in
AWS REGION	Asia Pacific (Mumbai) · ap-south-1
REPORT DATE	29 October 2026

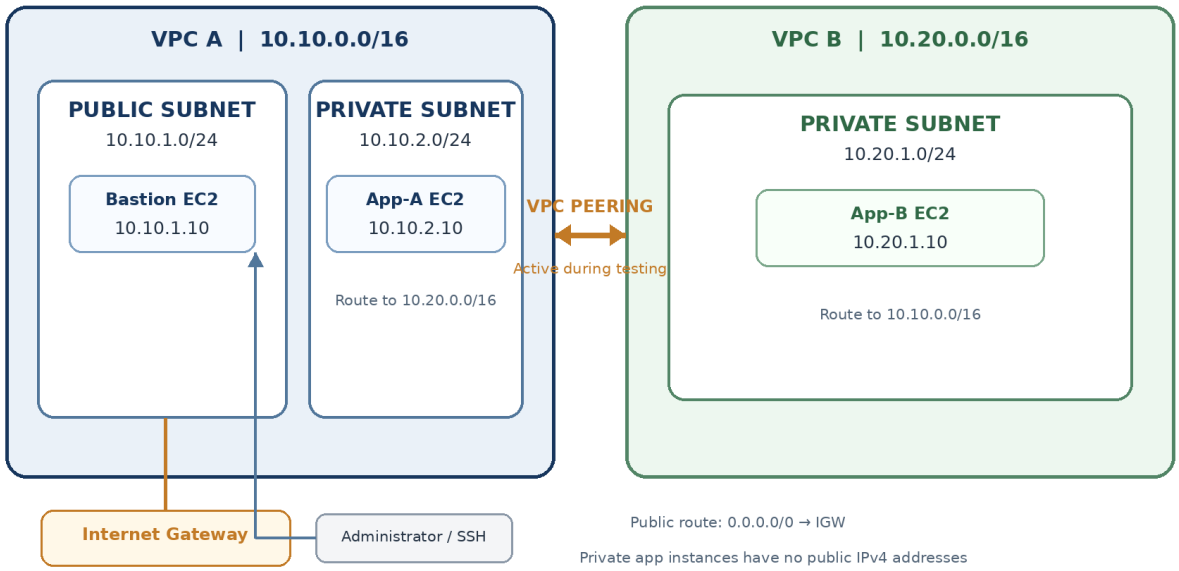
1. Project Objective

The objective of this project is to establish private network communication between two Amazon Virtual Private Clouds (VPCs) using VPC Peering. The project involves configuring non-overlapping VPC and subnet CIDRs, setting up public and private subnets, configuring route tables and security groups, and using a public Bastion host in VPC A for administration. The connectivity between the private application instances in both VPCs is tested using their private IPv4 addresses to verify successful communication, including forward and return traffic.

2. Network Architecture

The diagram below shows the intended network layout. The Bastion is in VPC A's public subnet, App-A is in VPC A's private subnet, and App-B is in VPC B's private subnet. The two VPCs communicate through the peering connection.

AWS VPC PEERING — NETWORK ARCHITECTURE



COMPONENT	CIDR / PRIVATE IP	PURPOSE
VPC A	10.10.0.0/16	First network
VPC A public subnet	10.10.1.0/24	Bastion host
VPC A private subnet	10.10.2.0/24	App-A
VPC B	10.20.0.0/16	Second network
VPC B private subnet	10.20.1.0/24	App-B
Bastion	10.10.1.10	Administrative entry point
App-A	10.10.2.10	Private workload
App-B	10.20.1.10	Private workload

3. Resource Inventory

Resource IDs are from the captured lab configuration. Resources were later removed during cleanup.

RESOURCE	NAME	RECORDED ID
VPC A	Aquib-ewb-vpc-a	vpc-09cce6b67e255934f
VPC B	Aquib-ewb-vpc-b	vpc-0fd3b4afe243071b1
Public subnet A	Aquib-ewb-a-public	subnet-0671e024ca9f41864
Private subnet A	Aquib-ewb-a-private	subnet-0031d4622dbfa7e22
Private subnet B	Aquib-ewb-b-private	subnet-0822fad27f09ba338
Internet Gateway	Aquib-ewb-igw-a	igw-09b6616b01bcd9b24
Public route table A	Aquib-ewb-rt-a-public	rtb-05a8068a98f7c3d67
Private route table A	Aquib-ewb-rt-a-private	rtb-0aa371a6f4b13ccc1
Private route table B	Aquib-ewb-rt-b-private	rtb-0f79689721a45217e
VPC Peering	Aquib-ewb-a-b-peer	pcx-0acc5a7e0500b3f28
Bastion SG	Aquib-ewb-sg-bastion	sg-0f21f44ef244b28d1
App-A SG	Aquib-ewb-sg-app-a	sg-000957832343bb393
App-B SG	Aquib-ewb-sg-app-b	sg-099c810bba12a44a1
Bastion EC2	ewb-bastion	i-005fedfaacda24293
App-A EC2	ewb-app-a	i-02019b5154ba19487
App-B replacement	ewb-app-b	i-07e3f063755fc90b5

4. Route Tables and How Traffic Moves

Route tables tell AWS where to send packets. Both VPCs need a route for the other VPC's CIDR. The public route table also needs a default route to the Internet Gateway for Bastion administration.

ROUTE TABLE	ROUTE	PURPOSE
VPC A public	0.0.0.0/0 → Internet Gateway	Bastion internet access
VPC A public	10.20.0.0/16 → Peering	Traffic to VPC B
VPC A private	10.20.0.0/16 → Peering	App-A to App-B
VPC B private	10.10.0.0/16 → Peering	Return traffic to VPC A

Forward path: App-A sends traffic to 10.20.1.10 through the peering connection. VPC B delivers it to App-B using its local route.

Return path: App-B replies to 10.10.2.10 through the VPC B peering route, and VPC A delivers the reply using its local route.

5. Security Group Rules

Security groups work like firewalls for EC2 network interfaces. The lab used specific source IPs rather than allowing SSH or ICMP from everywhere.

SECURITY GROUP	INBOUND RULE	SOURCE

Bastion	SSH TCP 22	Administrator public IP /32 (latest shown: 157.34.238.38/32)
App-A	SSH TCP 22	10.10.1.10/32 — Bastion
App-A	ICMP Echo Request	10.20.1.10/32 — App-B
App-B	SSH TCP 22	10.10.1.10/32 — Bastion
App-B	ICMP Echo Request	10.10.2.10/32 — App-A

6. Connectivity Tests

Connectivity was checked before and after the peering routes were added. The screenshots below record baseline failure, successful forward ping, private SSH and reverse ping.

TEST	RECORDED RESULT
Baseline App-A → App-B ping	4 sent, 0 received; 100% loss
Forward ping App-A → App-B	4 sent, 4 received; 0% loss
Private SSH via Bastion	Successful session recorded in earlier screenshot
Reverse ping App-B → App-A	4 sent, 4 received; 0% loss
Controlled ICMP failure/recovery	Not evidenced in supplied screenshots

7. Cleanup

After the tests, EC2 instances were terminated and the peering connection, custom security groups, Internet Gateway and VPC resources were deleted. The screenshots record these cleanup steps. EBS volumes, snapshots and final billing were not independently verified with a final screenshot.

8. Conclusion

The lab demonstrates how two non-overlapping VPCs can communicate privately through VPC Peering when both route tables and security groups are configured correctly. The recorded screenshots include successful forward and reverse ping tests and earlier private SSH evidence. The resources were later removed to end the lab.

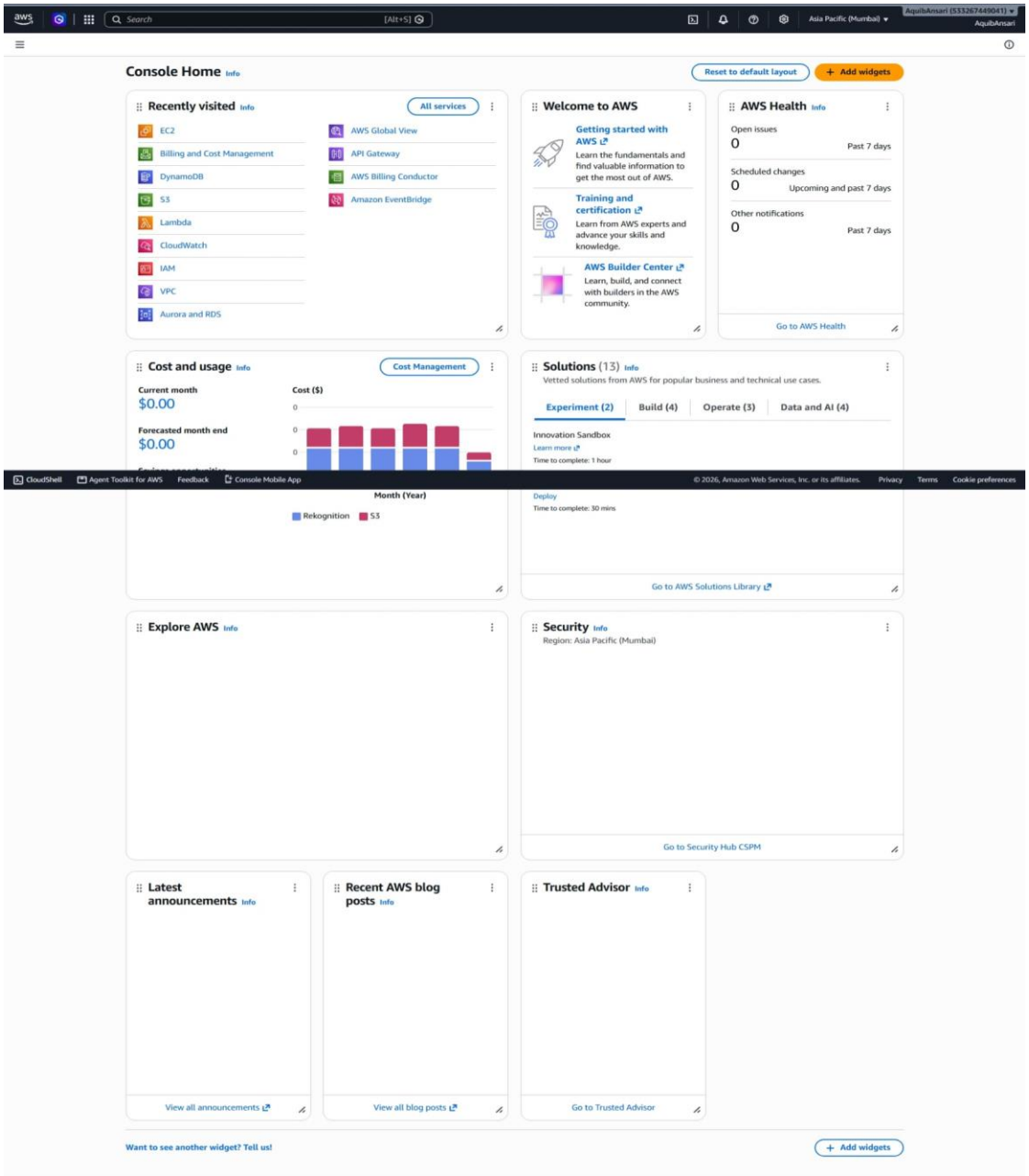
9. Step-by-Step Implementation Evidence

Each evidence item includes a simple explanation followed by the original screenshot from the lab.

Step 1 — Select the AWS Region

The project was configured in Asia Pacific (Mumbai), also known as ap-south-1. Keeping the lab resources in one region makes it easier to find the VPCs, subnets, route tables and EC2 instances in the same console.

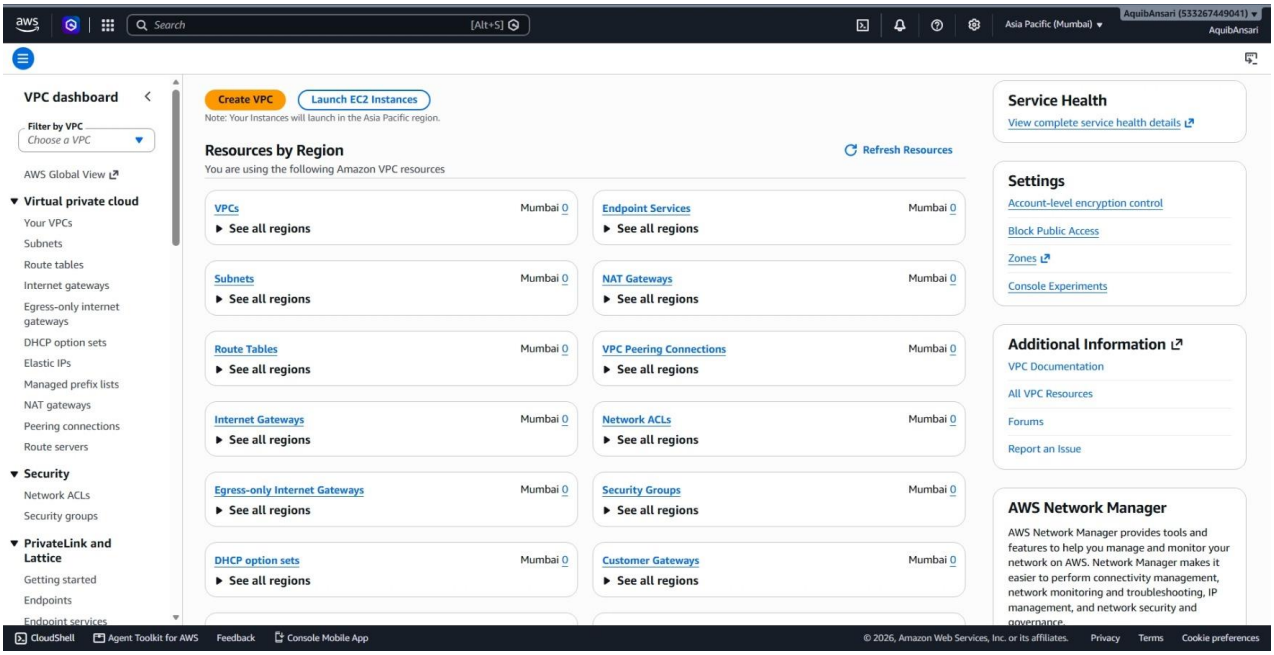
Evidence: AWS Region: Mumbai (ap-south-1).



Step 2 — Create VPC A

VPC A is the first isolated network. Its IPv4 CIDR is 10.10.0.0/16. This address range is separate from VPC B so that the two networks can communicate through VPC Peering without overlapping IP ranges.

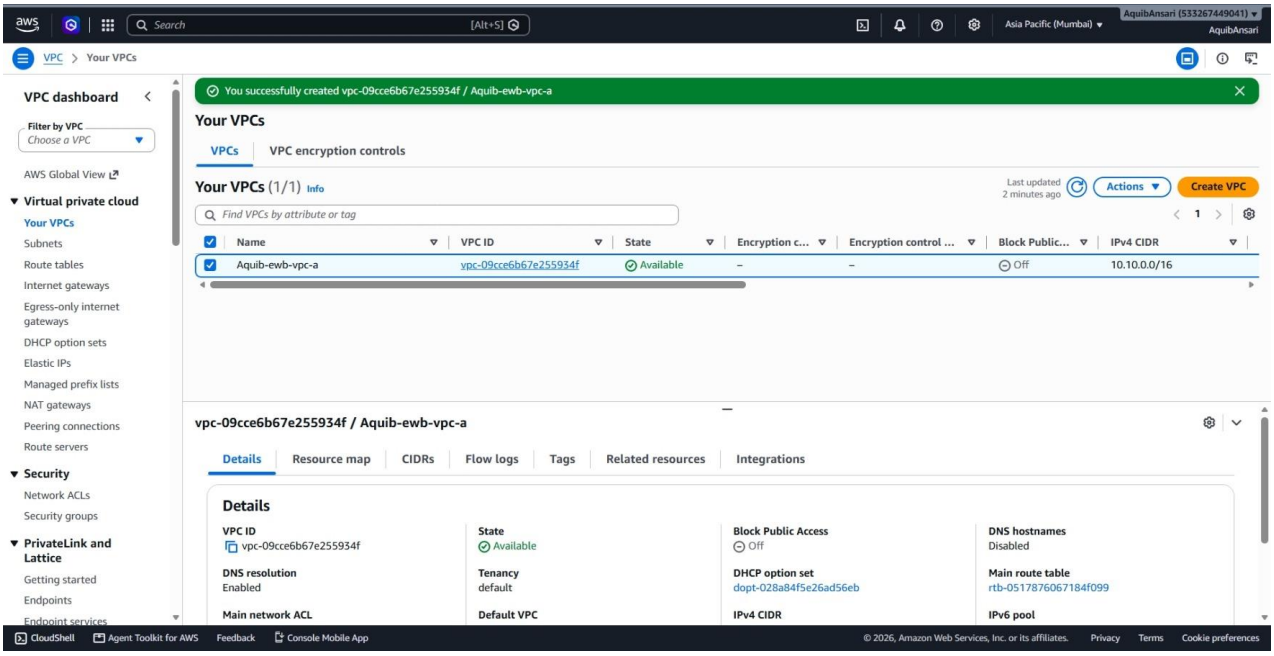
Evidence: VPC A configuration.



Step 3 — Configure VPC A

This screenshot records the first VPC and its network settings. VPC A contains the public subnet for the Bastion host and a private subnet for App-A.

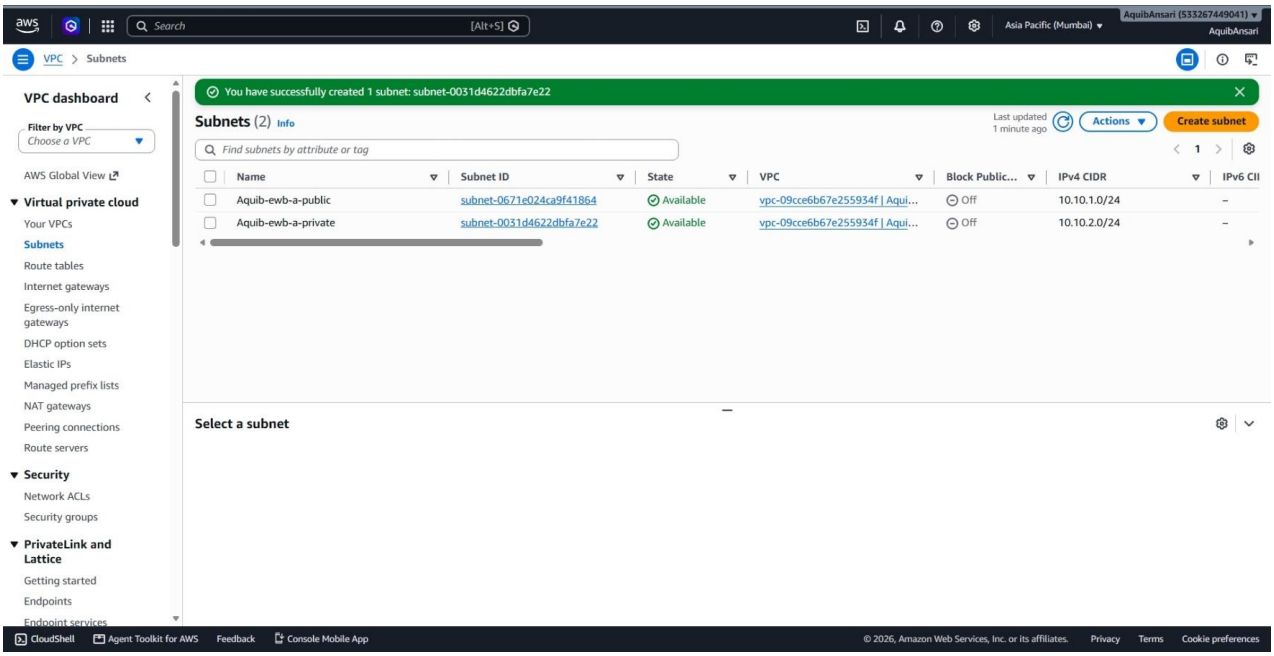
Evidence: VPC A settings.



Step 4 — Create the subnets

The lab uses a public subnet (10.10.1.0/24) for the Bastion and a private subnet (10.10.2.0/24) for App-A. VPC B has its own private subnet (10.20.1.0/24) for App-B.

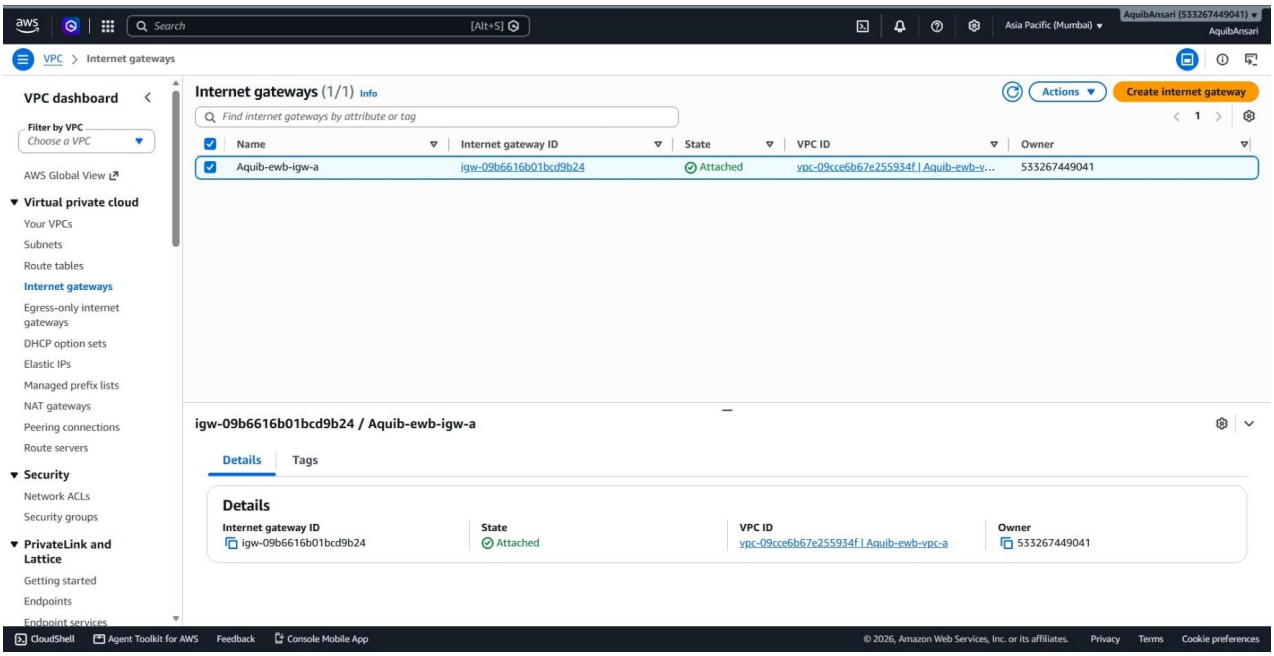
Evidence: Subnet configuration.



Step 5 — Attach an Internet Gateway

The Internet Gateway is attached to VPC A. It is used by the public subnet's route table so that the Bastion can be reached from the administrator's workstation, subject to the route table and security-group rules.

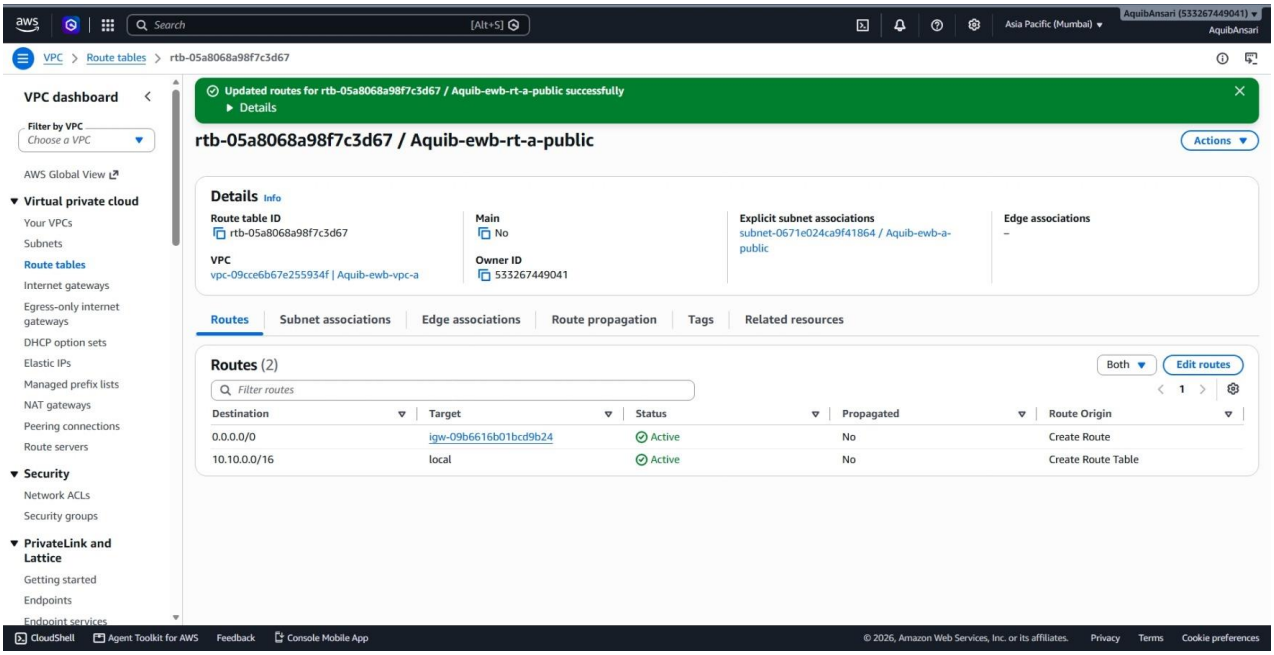
Evidence: Internet Gateway configuration.



Step 6 — Configure the public route table

The public route table is associated with VPC A's public subnet. It includes a default route to the Internet Gateway and a specific route for VPC B's CIDR through the peering connection. The more-specific VPC B route directs private VPC-to-VPC traffic through peering.

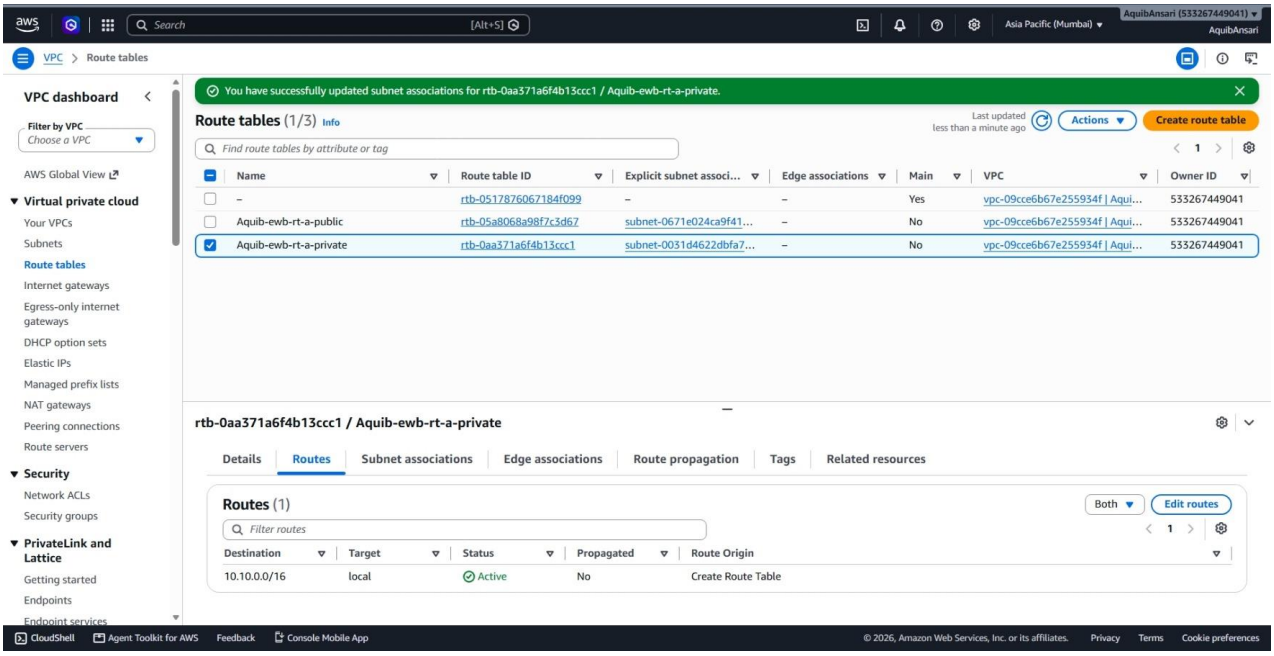
Evidence: Public route table and subnet association.



Step 7 — Configure VPC A's private route table

The private route table is associated with App-A's subnet. The route to 10.20.0.0/16 points to the peering connection, allowing App-A to send packets toward App-B's private address.

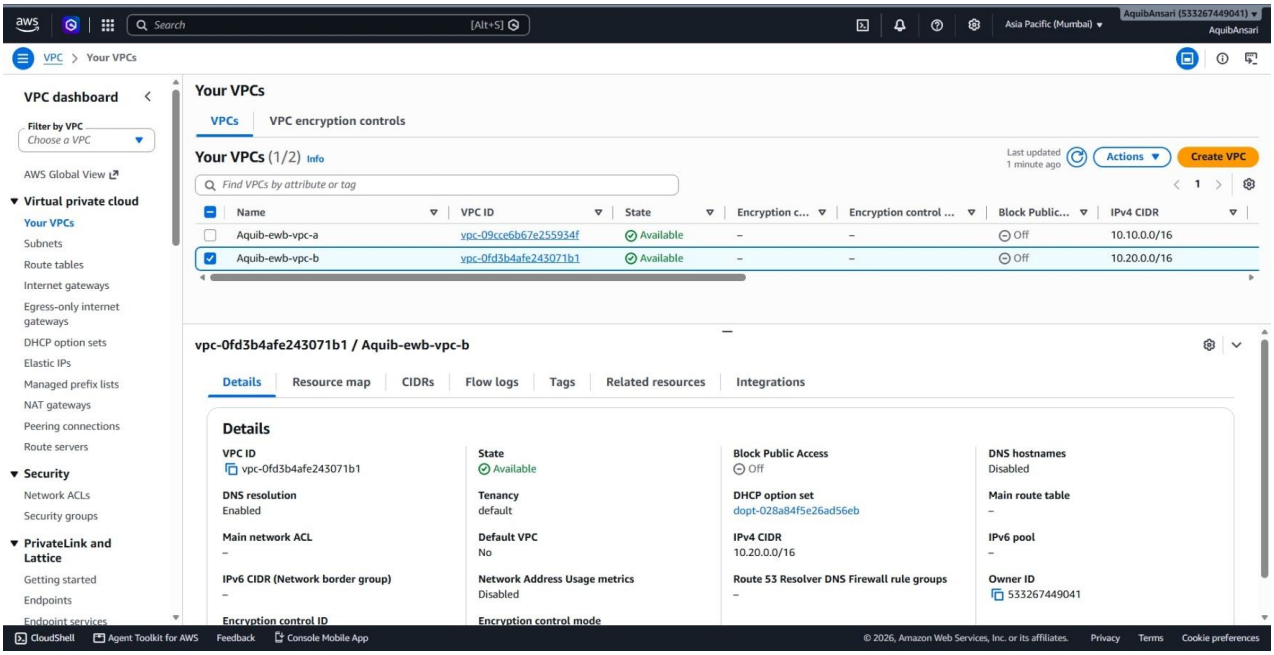
Evidence: VPC A private route table.



Step 8 — Create VPC B

VPC B is the second isolated network and uses 10.20.0.0/16. This range does not overlap with VPC A's 10.10.0.0/16 range.

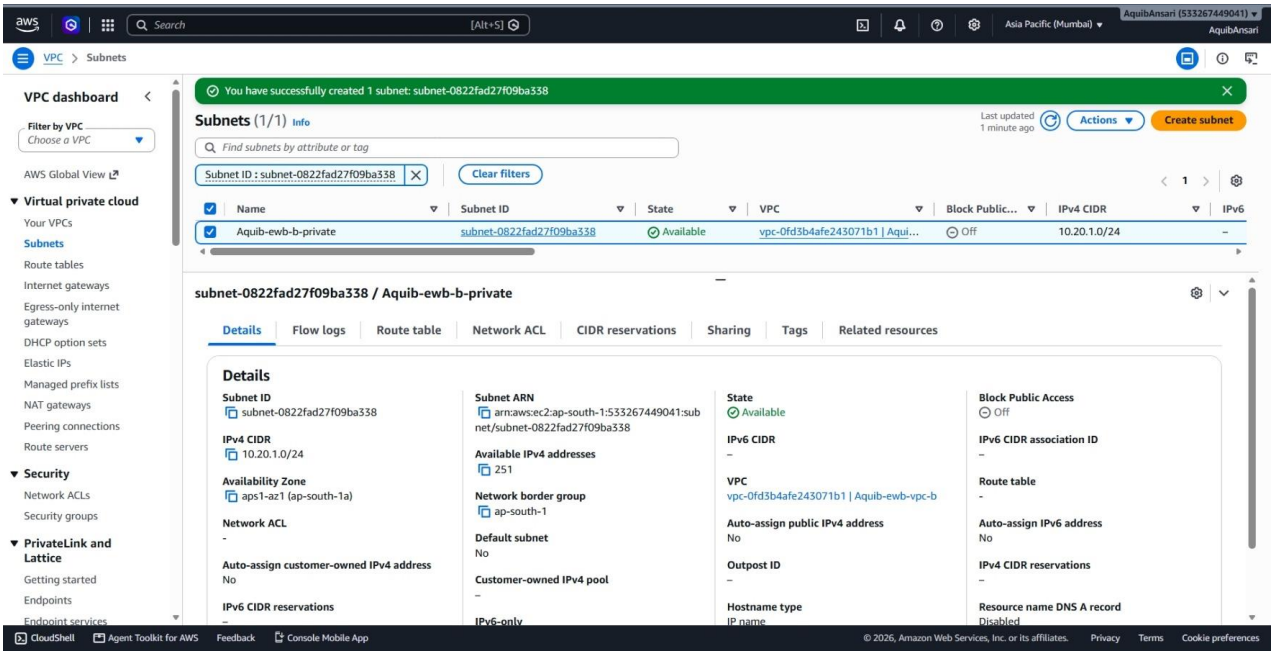
Evidence: VPC B configuration.



Step 9 — Create VPC B's private subnet

The subnet 10.20.1.0/24 hosts App-B. The instance is intended to have no public IPv4 address, so it is reached through private routing rather than directly from the internet.

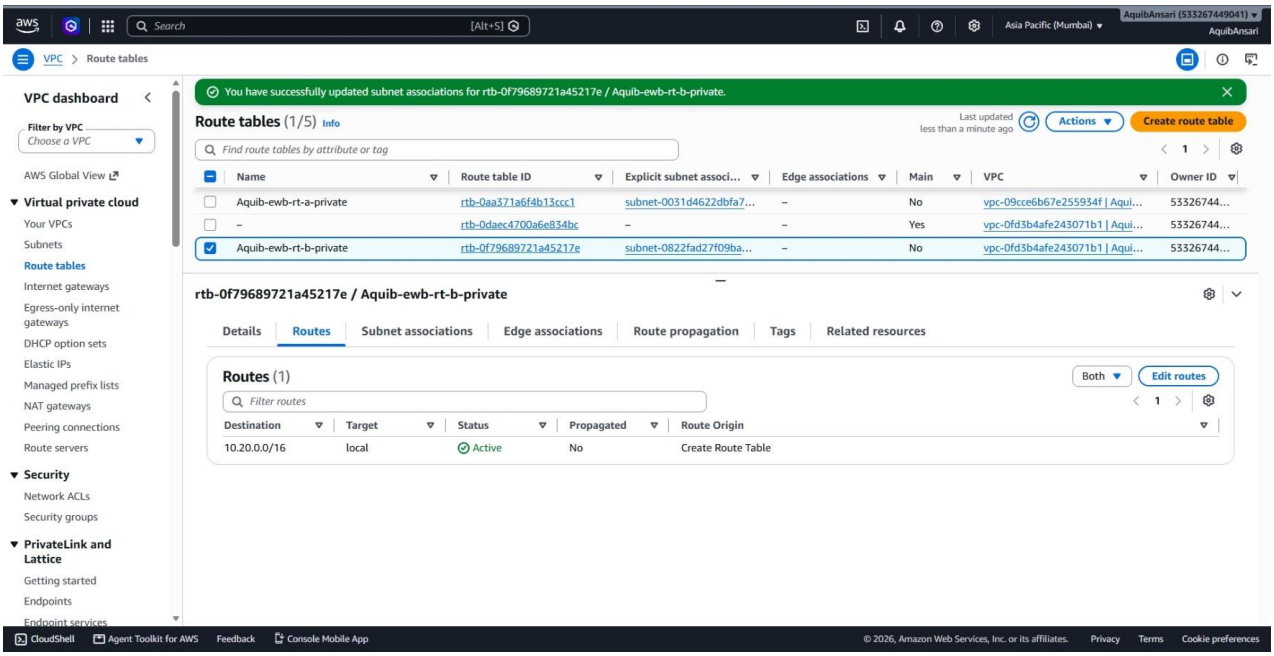
Evidence: VPC B private subnet.



Step 10 — Add the return route in VPC B

VPC B's private route table contains a route for 10.10.0.0/16 through the peering connection. This return route is necessary because replies from App-B must know how to reach App-A in VPC A.

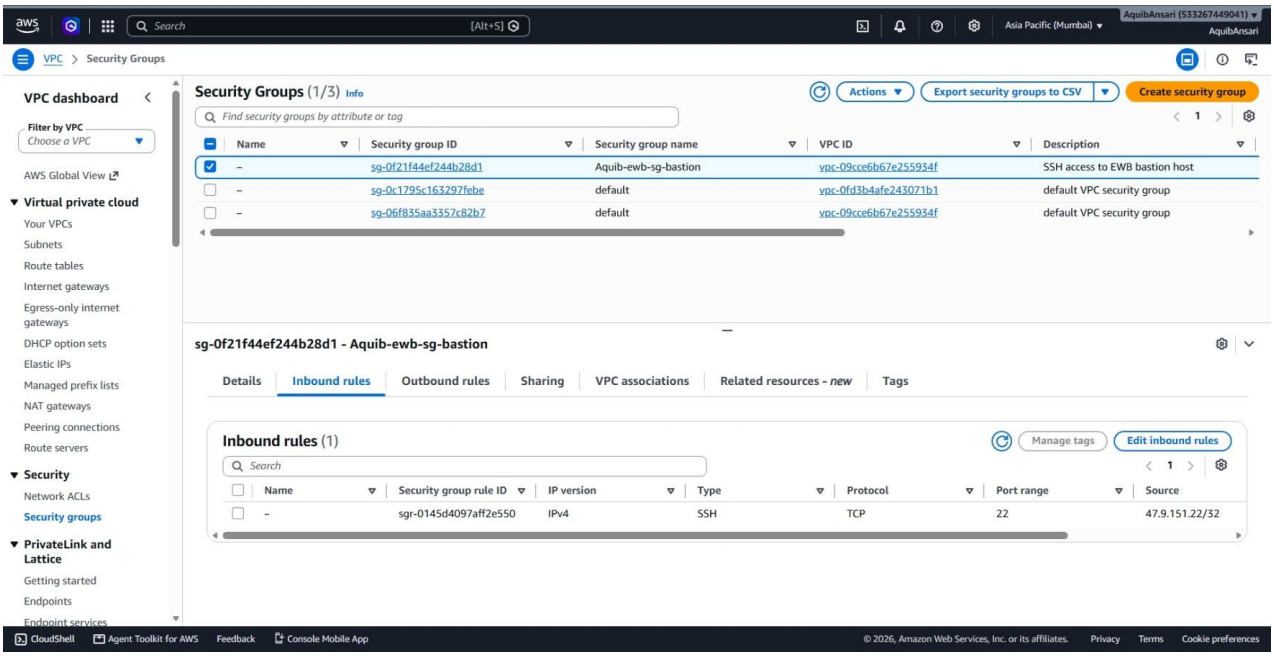
Evidence: VPC B private route table.



Step 11 — Restrict access to the Bastion

The Bastion security group allows SSH on TCP port 22 from the administrator's public IP address (/32). Restricting the source to one address is safer than allowing SSH from every IPv4 address.

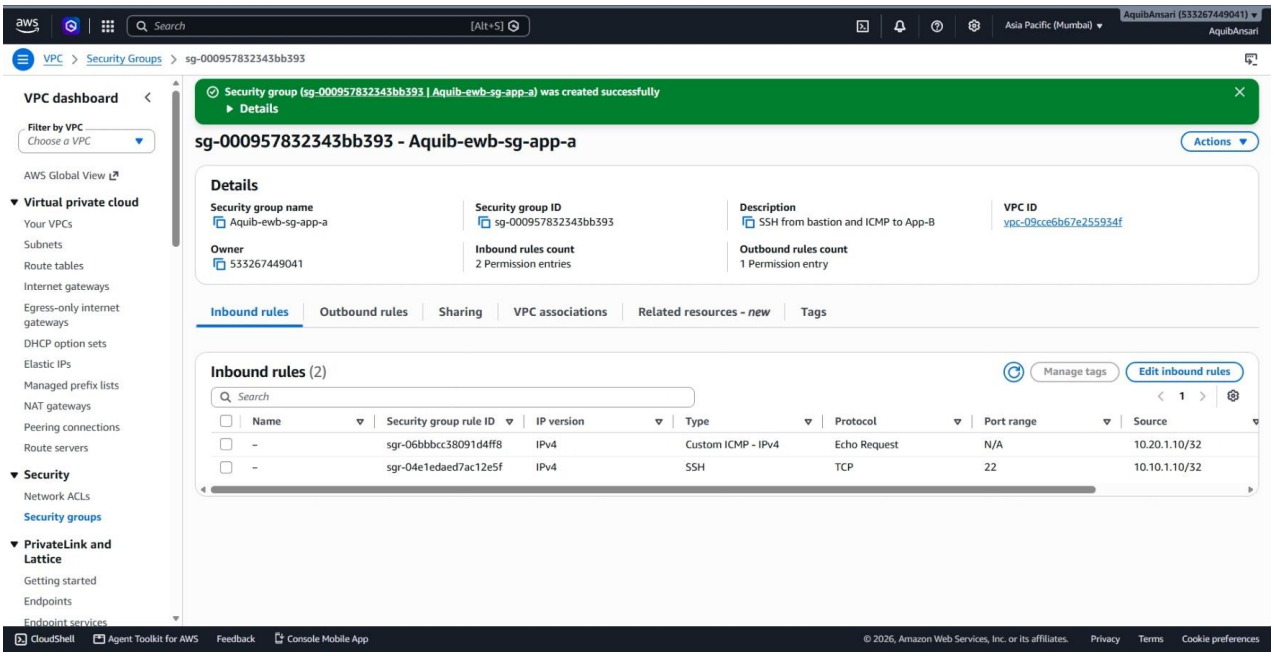
Evidence: Bastion security group.



Step 12 — Configure App-A's security group

App-A allows SSH from the Bastion's private address (10.10.1.10/32) and ICMP Echo Request from App-B (10.20.1.10/32). These rules limit access to the specific lab hosts that need to communicate.

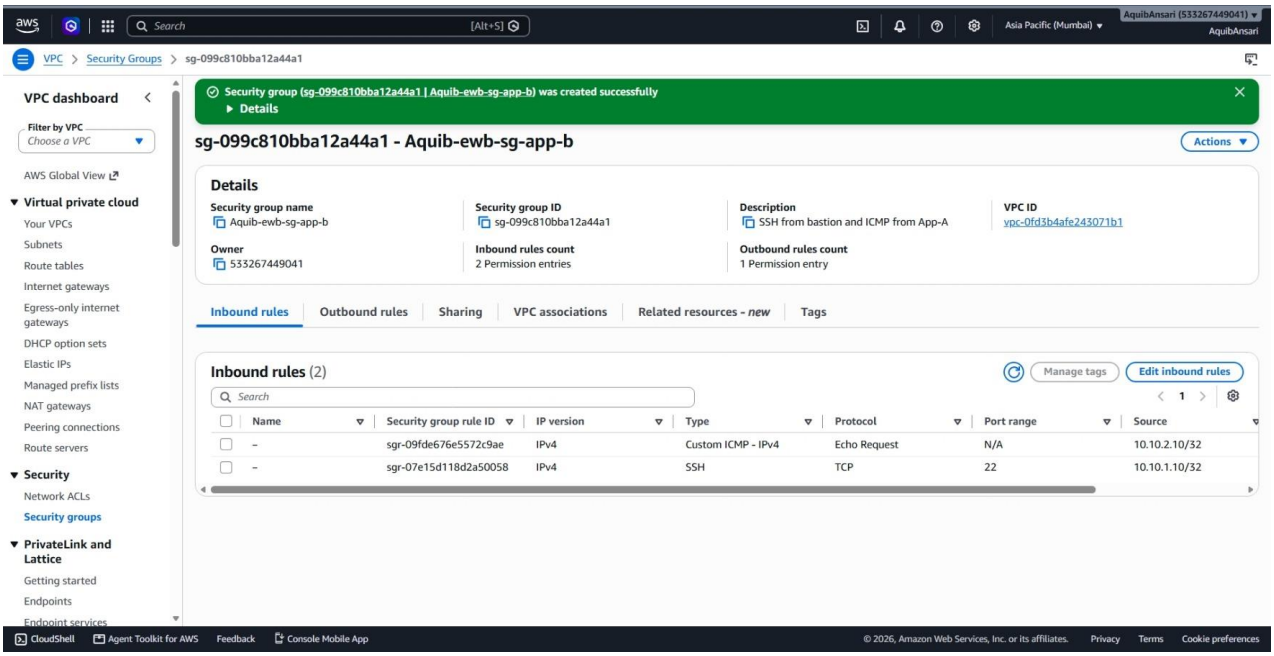
Evidence: App-A security group.



Step 13 — Configure App-B's security group

App-B allows SSH from the Bastion (10.10.1.10/32) and ICMP Echo Request from App-A (10.10.2.10/32). The source restrictions are important for testing the private peering path safely.

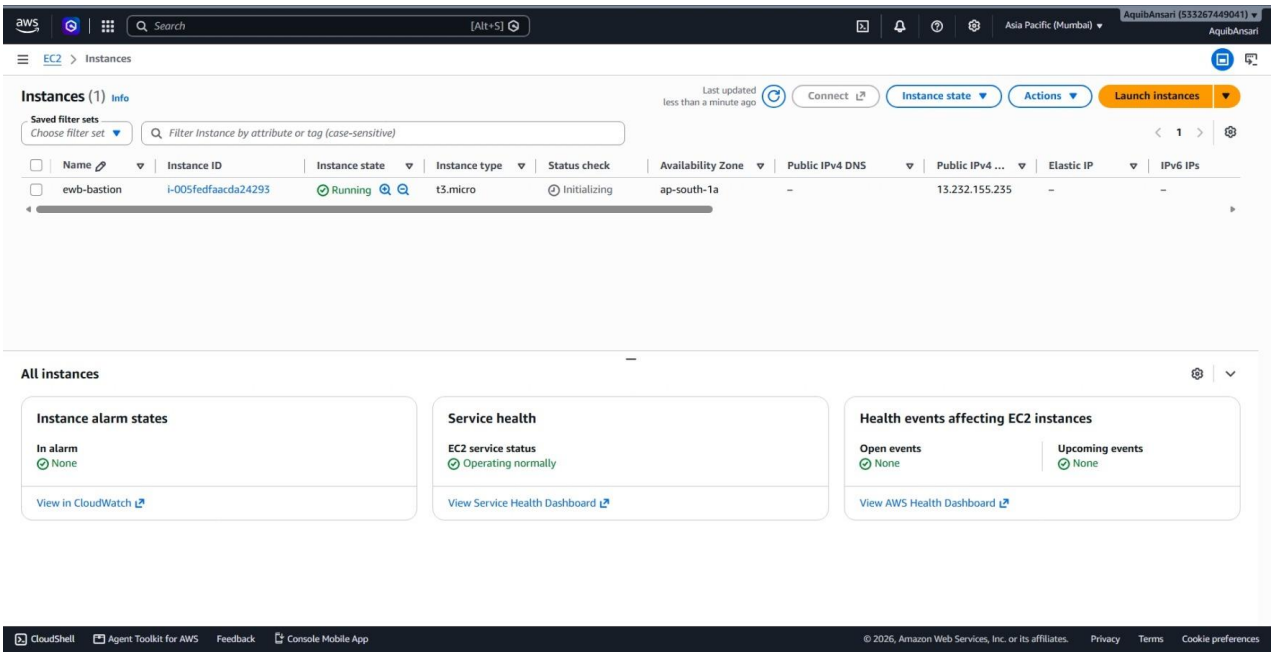
Evidence: App-B security group.



Step 14 — Launch the Bastion instance

The Bastion instance is placed in VPC A's public subnet with private IP 10.10.1.10. Its public IPv4 address is used only for the initial administrative connection; the private instances are not exposed with public IPs.

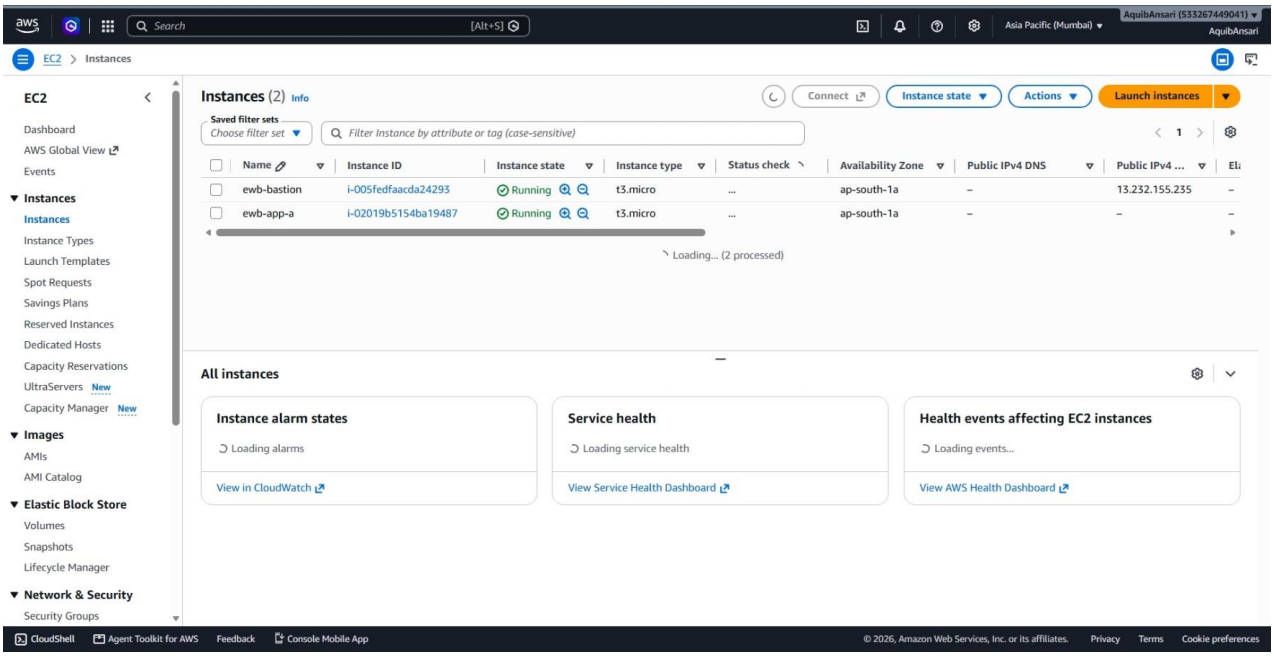
Evidence: Bastion EC2 instance.



Step 15 — Launch App-A in the private subnet

App-A uses private IP 10.10.2.10 in VPC A's private subnet. It is meant to be accessed through the Bastion or other approved private paths, not by direct public SSH.

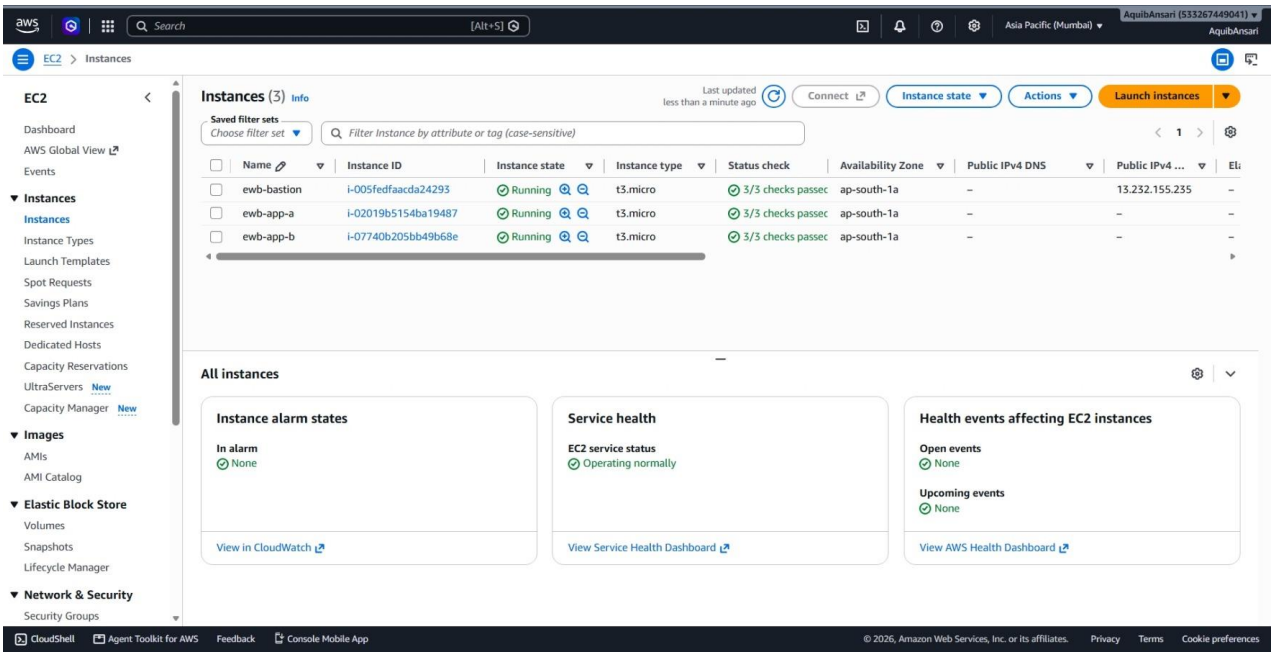
Evidence: App-A EC2 instance.



Step 16 — Launch App-B in VPC B

App-B uses private IP 10.20.1.10 in VPC B's private subnet. A replacement instance was launched during troubleshooting to correct its private IP address; the resource IDs in the report are historical records from the lab.

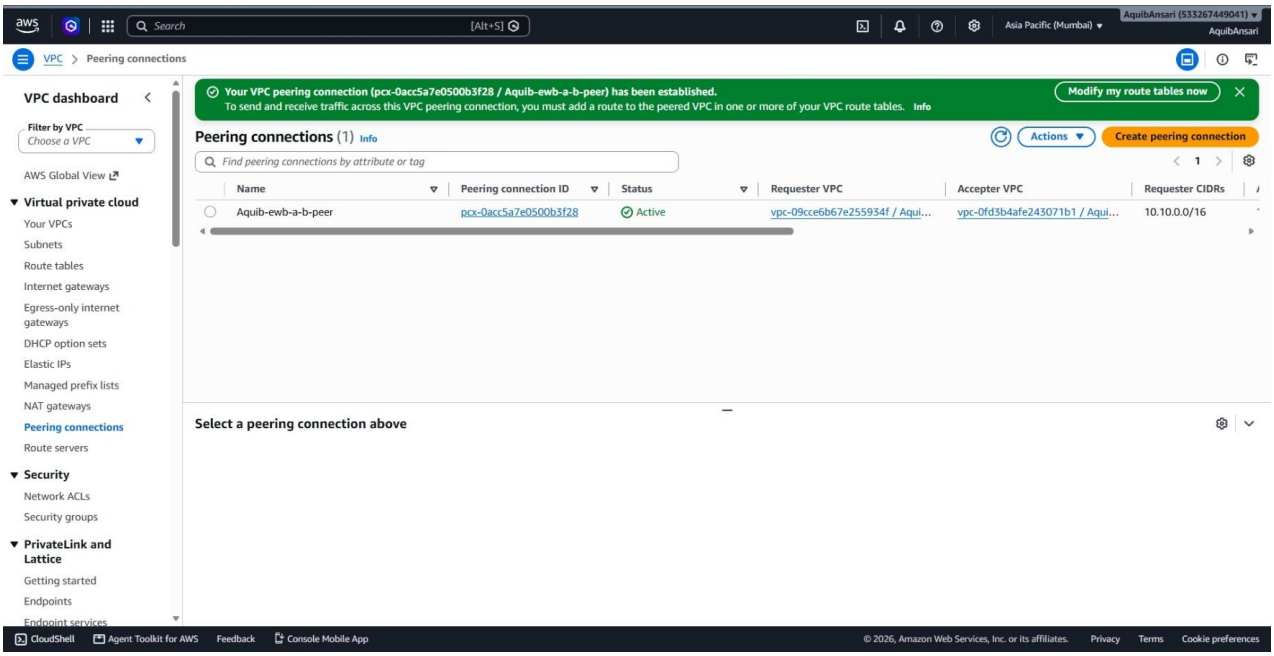
Evidence: App-B EC2 instance evidence.



Step 17 — Establish VPC Peering

The peering connection links VPC A and VPC B. An Active status means AWS has accepted the connection, but routing and security-group rules are still required before the instances can communicate.

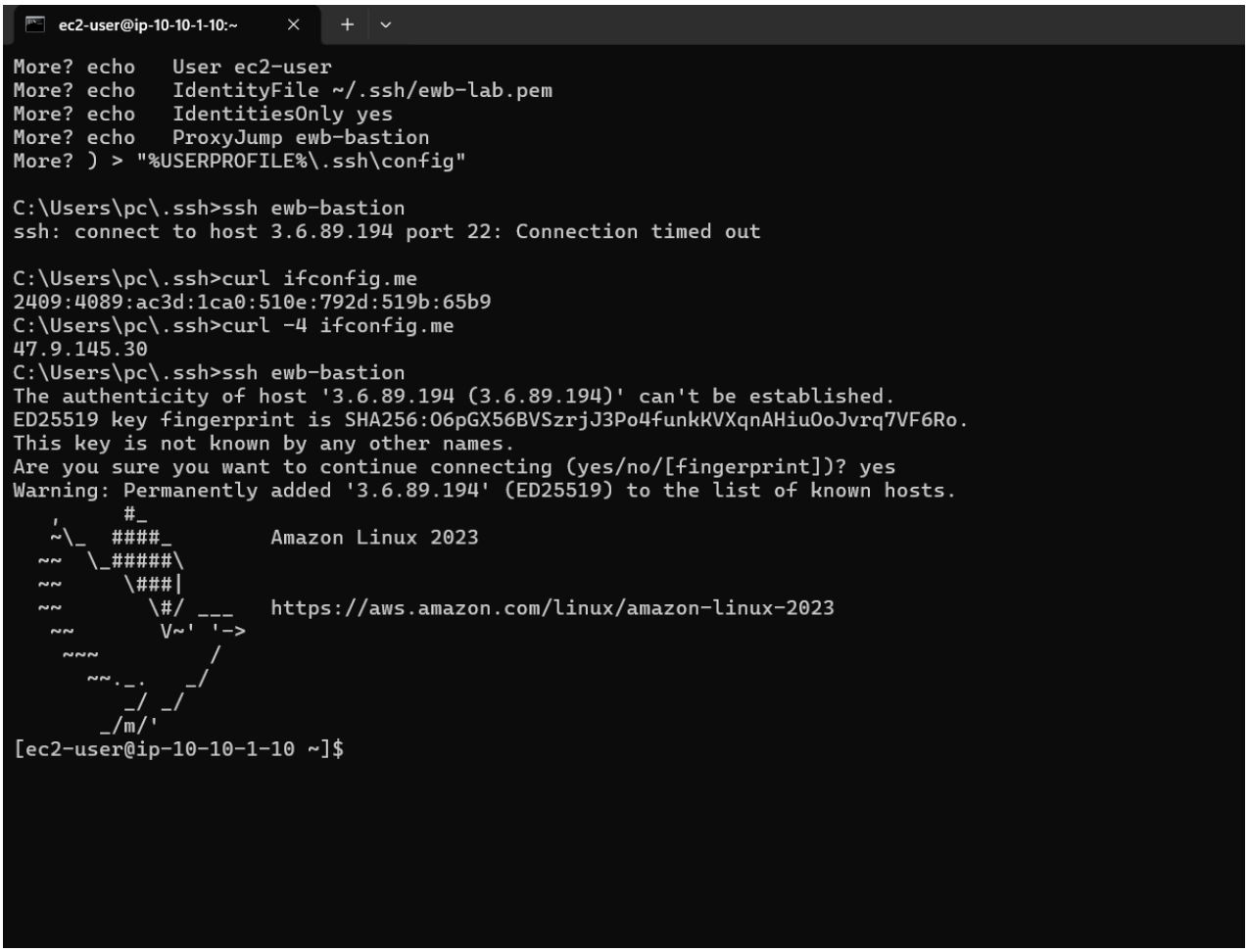
Evidence: Active VPC Peering connection.



Step 18 — Verify Bastion access

This screenshot records a previously successful login to the Bastion. It is included as historical evidence of the access path; a later browser-based EC2 Instance Connect attempt failed during troubleshooting.

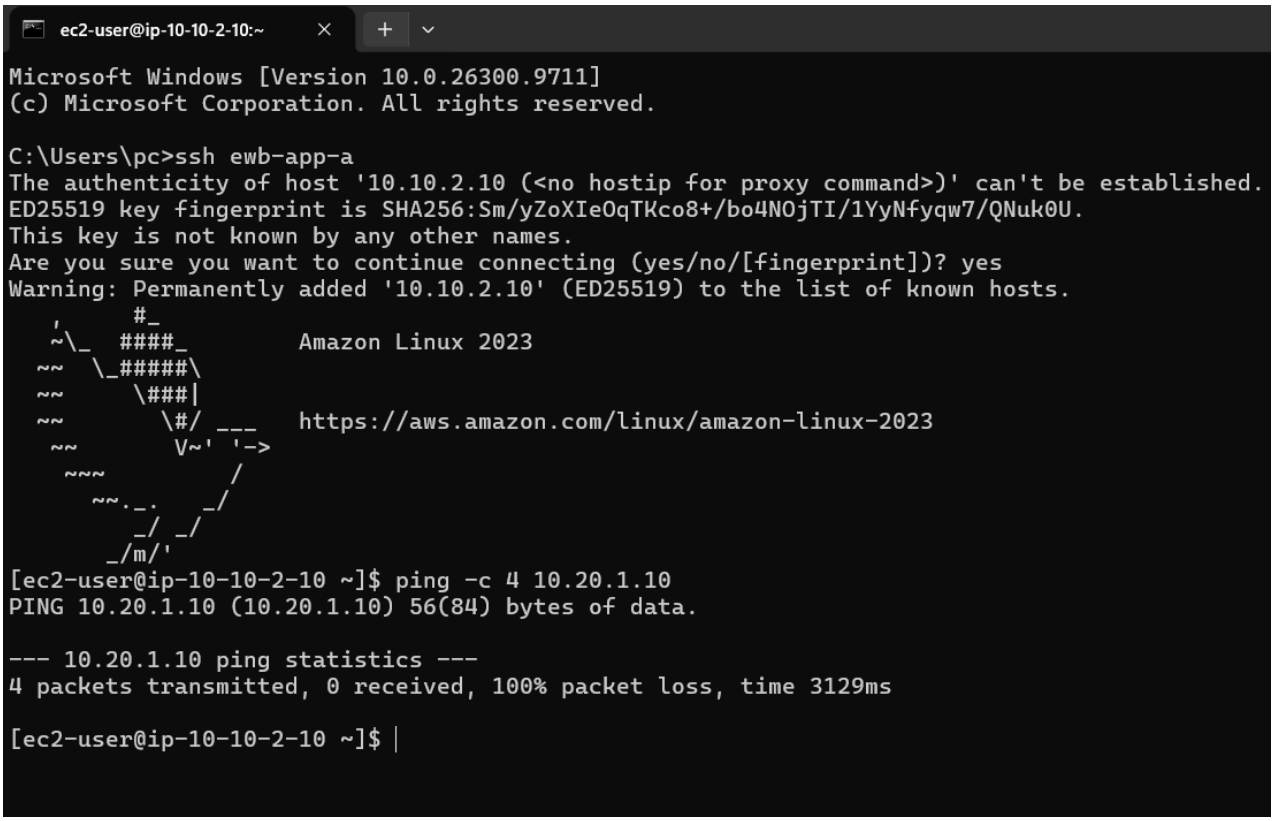
Evidence: Bastion login evidence.



Step 19 — Test connectivity before peering routes

Before the peering route setup, App-A pinged 10.20.1.10 and received no replies. This baseline failure shows that the two isolated VPCs could not communicate before the route and peering configuration was completed.

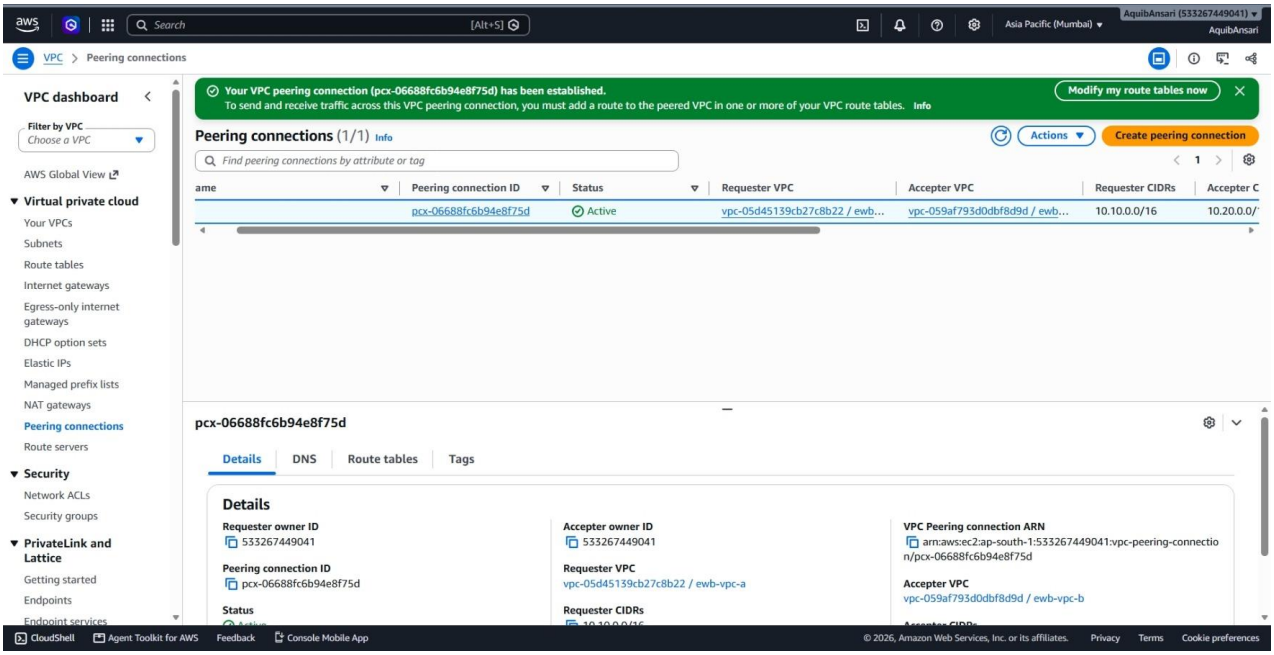
Evidence: Baseline ping test.



Step 20 — Review peering and routing

This evidence records the peering/routing configuration stage. Both VPCs need a route to the other VPC's CIDR through the same peering connection for bidirectional traffic.

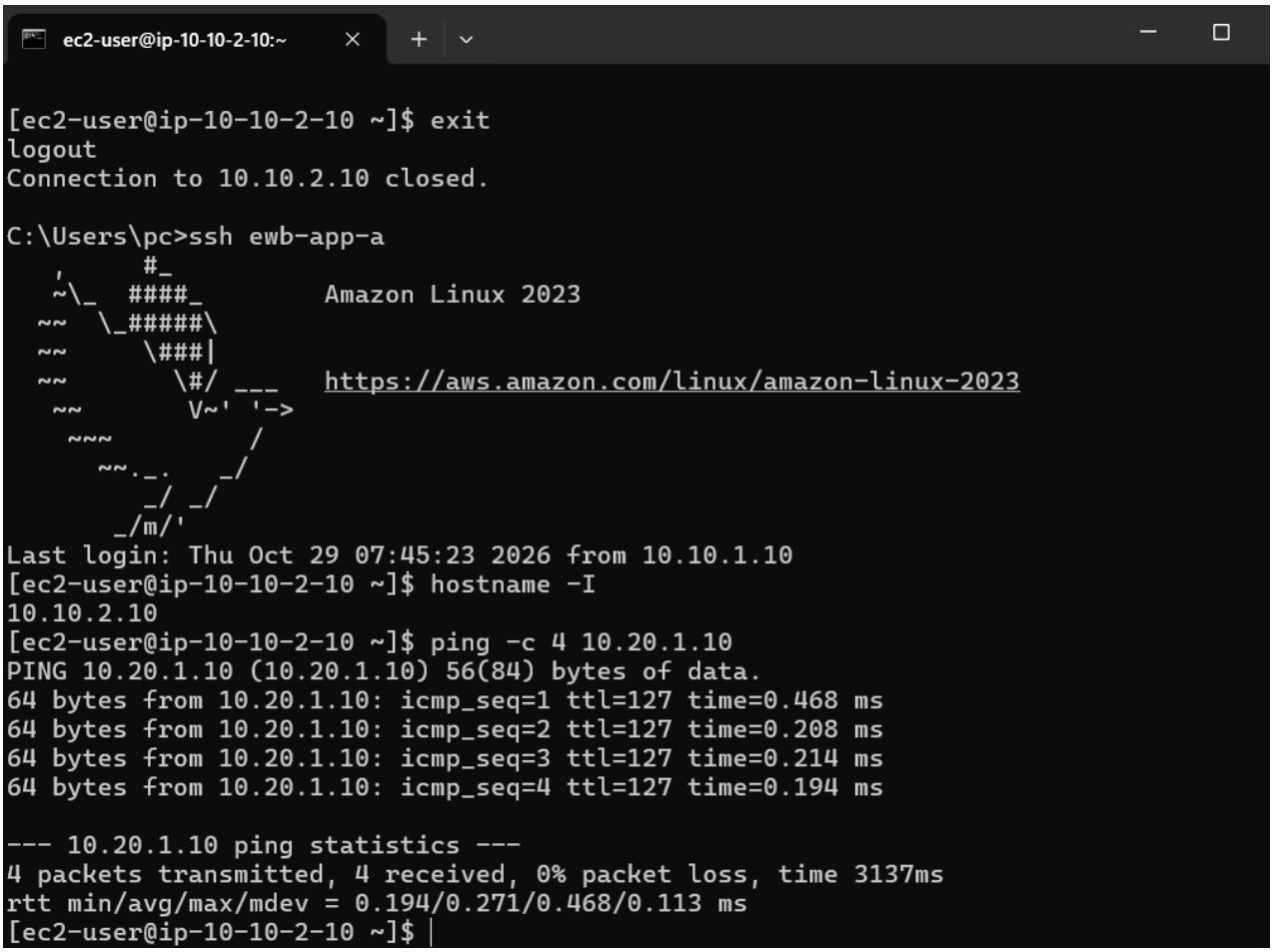
Evidence: VPC Peering and route evidence.



Step 21 — Check ping connectivity

This screenshot is part of the connectivity-test evidence. The report records the successful forward and reverse ping results from the original lab screenshots, where four packets were sent and four replies were received.

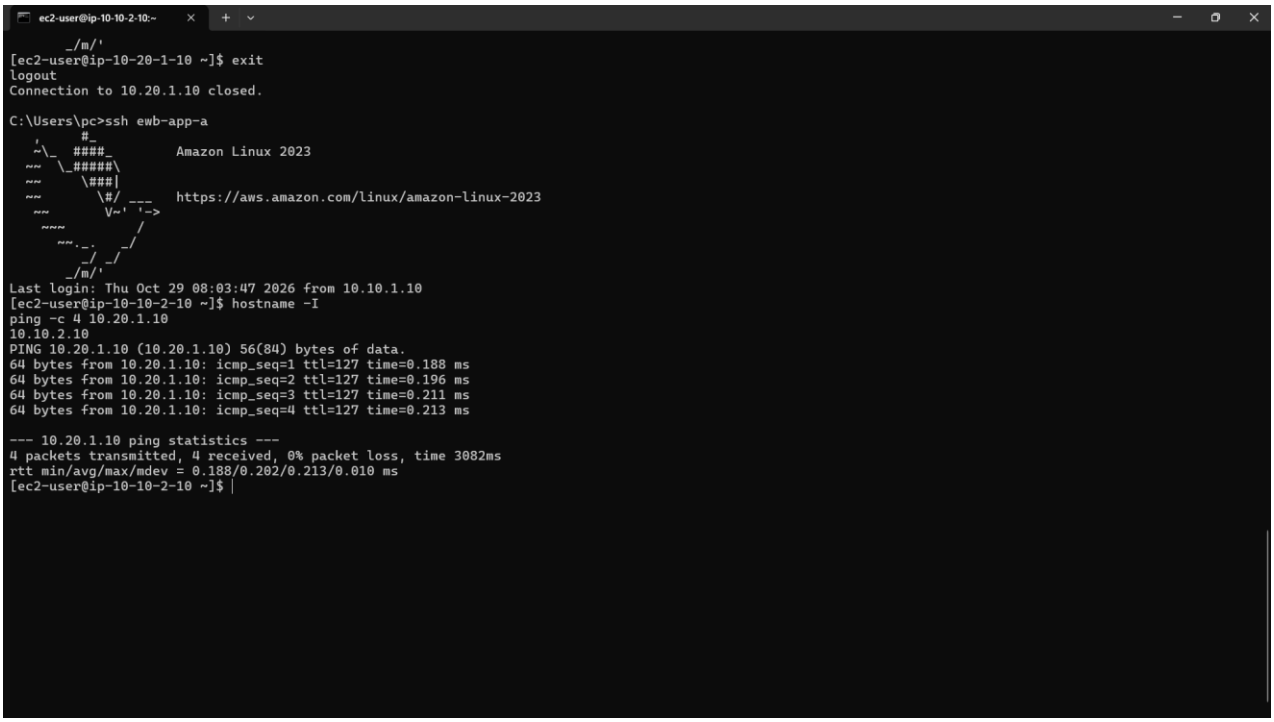
Evidence: Ping test output.



Step 22 — Verify private SSH and forward ping

This terminal evidence shows private-host access and a forward ping from App-A to App-B. The successful ping output records 4 packets transmitted, 4 received and 0% packet loss.

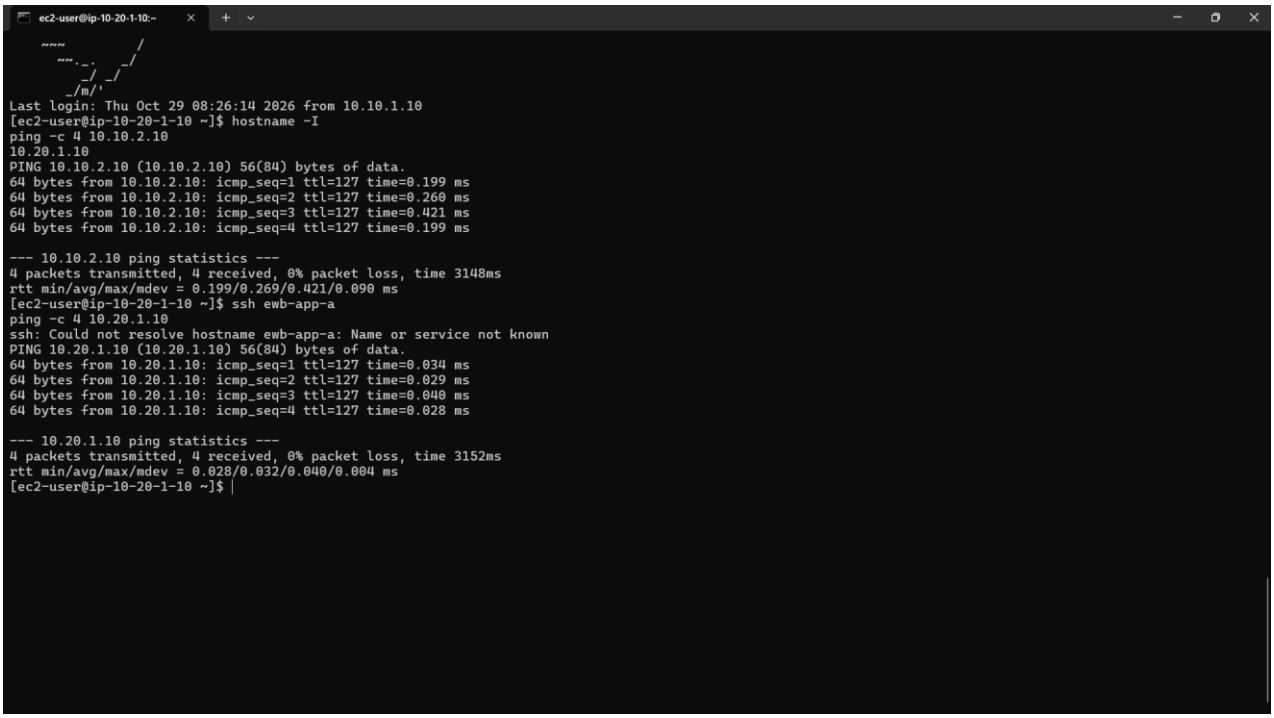
Evidence: Private SSH / forward ping evidence.



Step 23 — Verify the reverse path

From App-B, the ping to App-A's private address 10.10.2.10 succeeded. This confirms that the return route from VPC B to VPC A was present during the recorded test.

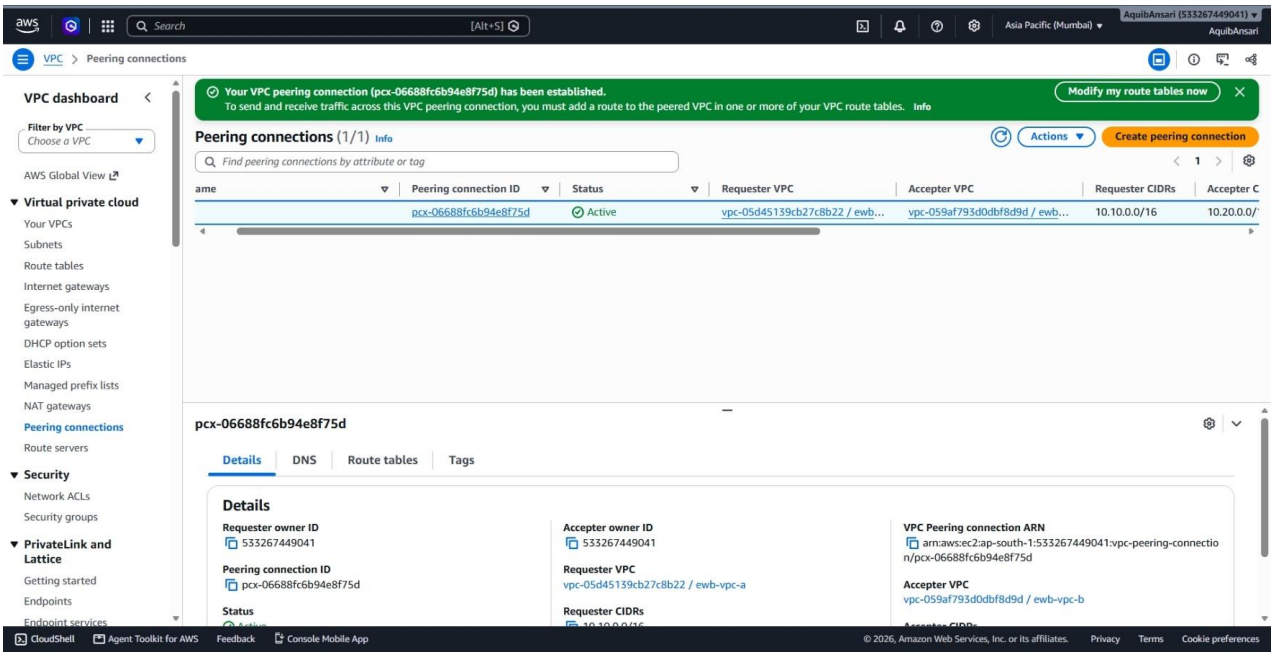
Evidence: Reverse ping from App-B to App-A.



Step 24 — Capture the peering connection

This screenshot records creation of the VPC Peering connection during the lab. It supports the resource-setup history before the final cleanup.

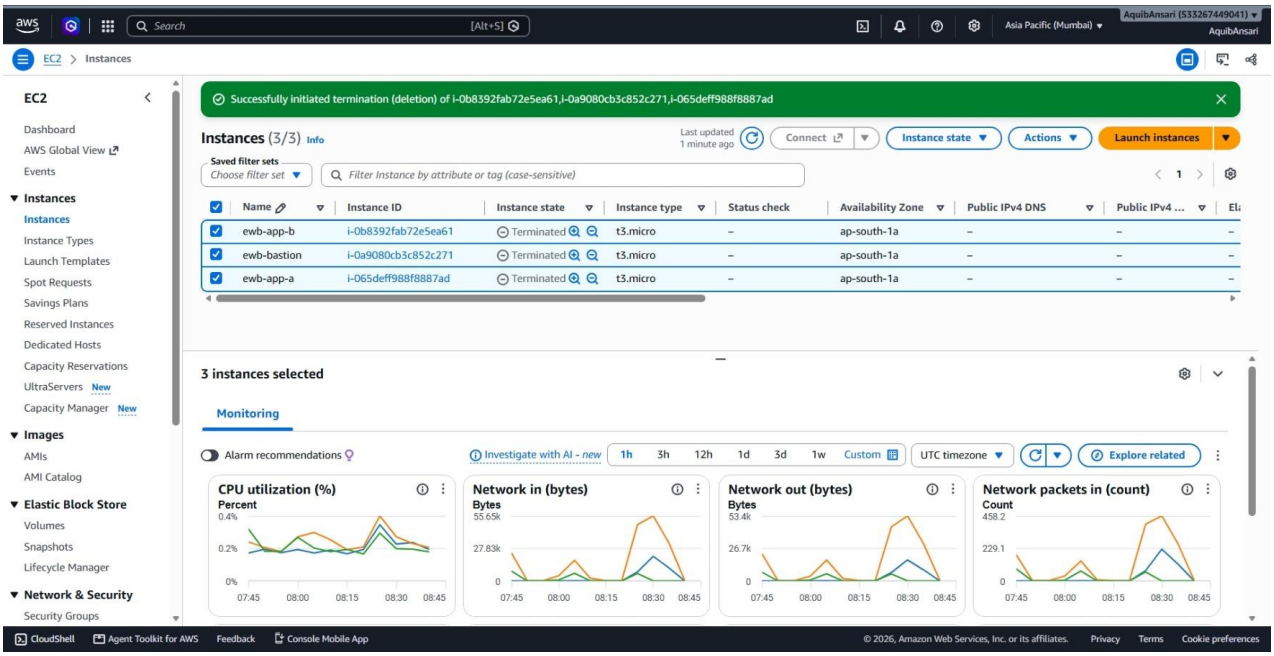
Evidence: Peering creation evidence.



Step 25 — Terminate EC2 instances

The EC2 instances were terminated during cleanup to stop compute usage. The lab's resources were removed after the network tests were completed.

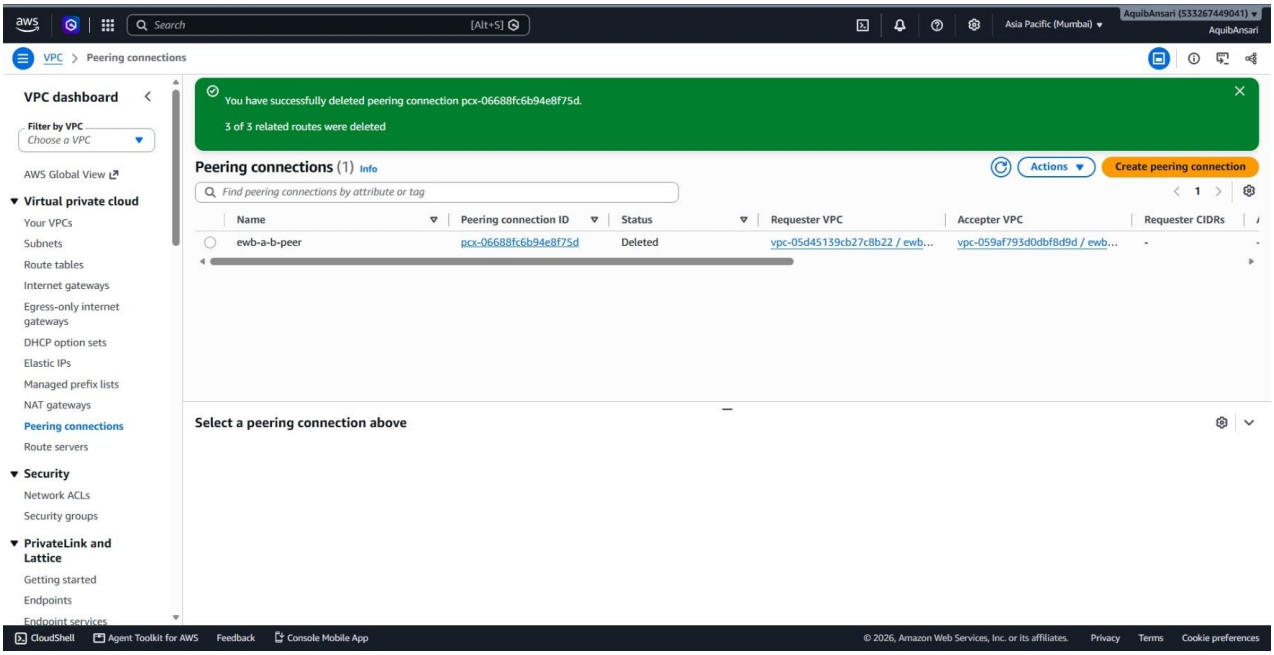
Evidence: EC2 termination evidence.



Step 26 — Delete the peering connection

After the tests, the VPC Peering connection was deleted. This removes the connection between the two lab VPCs and helps ensure the environment is torn down.

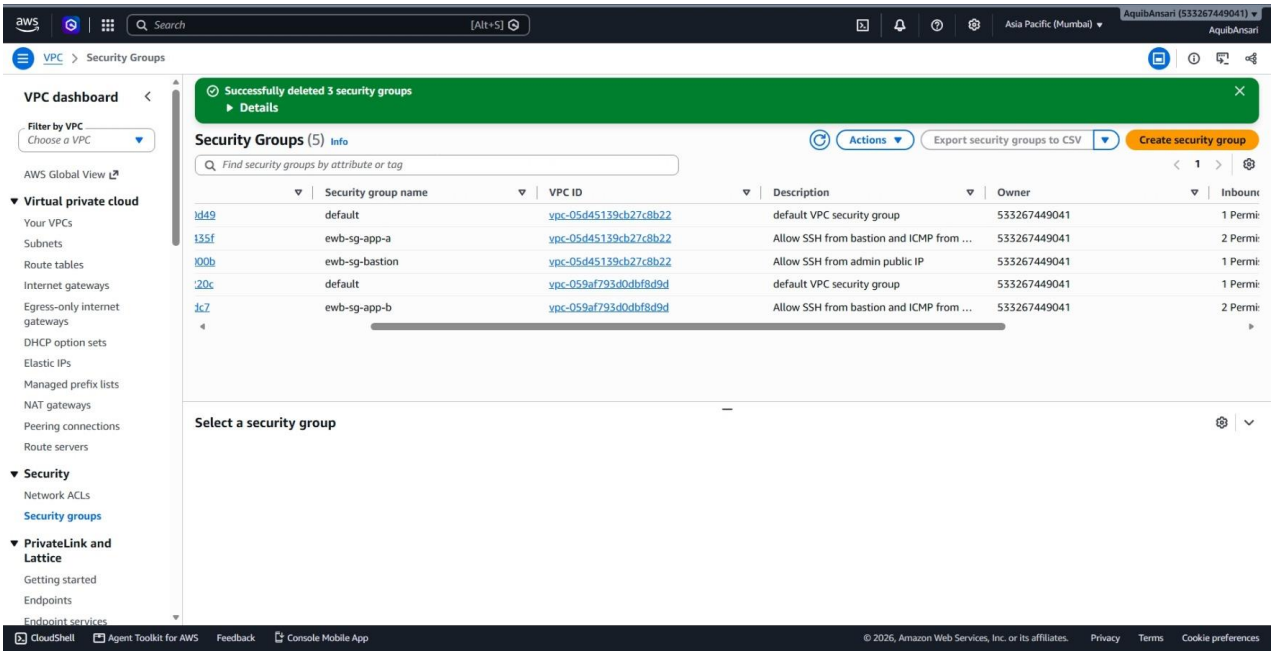
Evidence: Peering deletion evidence.



Step 27 — Delete custom security groups

The custom security groups were deleted after the associated instances/network interfaces no longer depended on them.

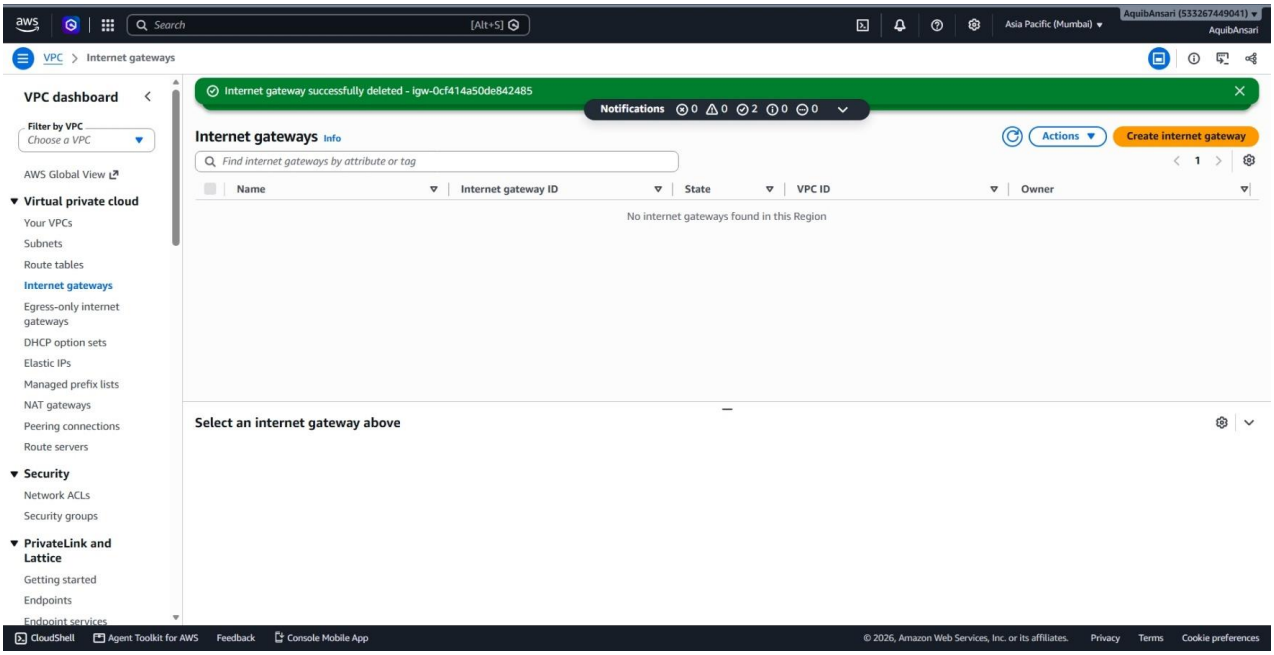
Evidence: Security-group cleanup evidence.



Step 28 — Delete the Internet Gateway

The Internet Gateway was deleted as part of removing the public access path for the lab network.

Evidence: Internet Gateway cleanup evidence.



Step 29 — Delete the VPC

The final screenshot records VPC cleanup. The lab's custom VPC resources were removed after the EC2 instances and dependent network resources were deleted.

Evidence: VPC deletion evidence.

